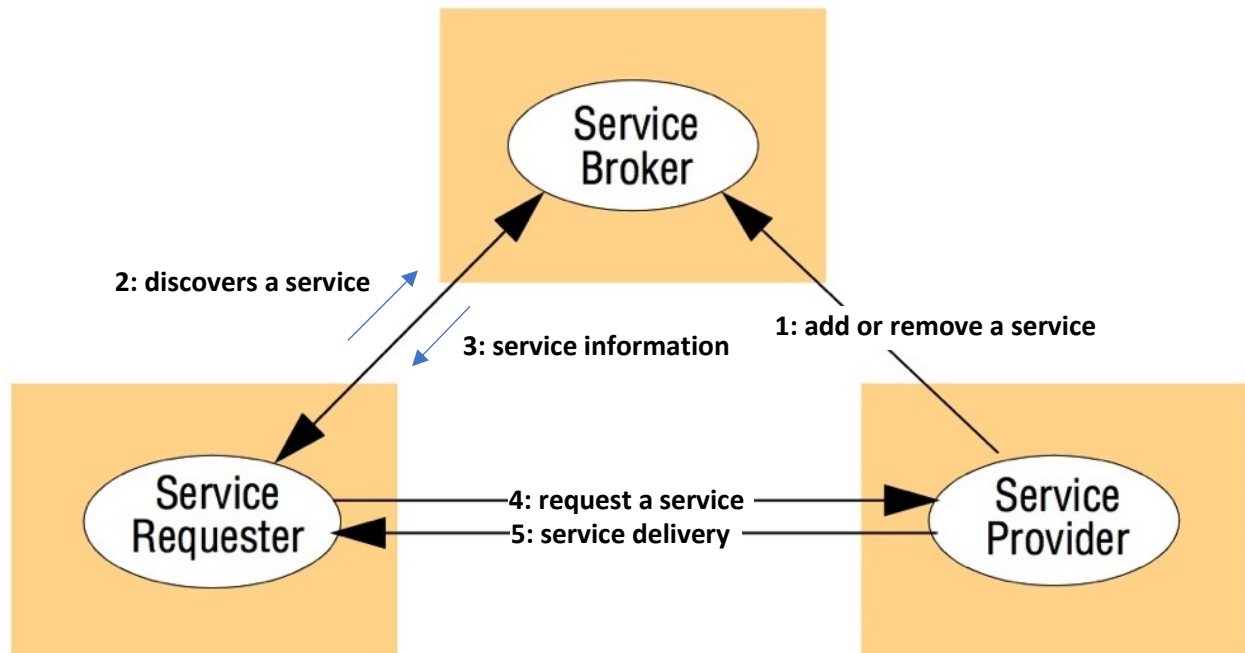


Development project

NOTES: This project is a two-person team project.

Project Objectives: This project provides hands-on experience of developing distributed systems, in particular a system based on the service brokerage pattern. More discussions about brokers can be found in the textbook, in particular Chapter 2 (System Models) and Chapter 6 (Indirect Communication).



Description: As shown in Figure 1, there exist three types of entities in the system.

- A **service provider** provides services, which may be requested by clients (that is, requesters).
- A **service requester** may request a service from a service provider.
- The **service broker** is in charge of keeping an inventory of services published by service providers and, when requested by a service requester, provides related service information in response to the request.

This project has two parts as described below.

Part A

Each team should develop the following programs.

- The **service broker**:
 - The broker uses a well-known port, which is to be used by both the service providers and the service requesters when communicating with the service broker.

- The broker should support the service **providers** by allowing them to send `addService()` and `removeService()` requests to the broker. The `addService()` request from a service provider causes the broker to add the name of the service, the IP address of the provider, and the port number to be used for that service. The `removeService()` request causes the broker to find and remove the named service from its directory of services.
 - The broker should support the service **requesters** by allowing them to send `discoverService()` requests, which include a key phrase of the requested service. The broker should look up services in the service directory using the specified key phrase, and return the service provider that best matches that key phrase; if no matching service is found, it returns an error code to the requester.
2. Service provider #1: The **randomNumberGenerator** provider:
 - This provider supplies a random number to the requester.
 3. Service provider #2: The **hashGenerator** provider:
 - This provider takes a string (for example, “How are you doing?”) and the name of a hash method (for example, MD5) as the input from the client and returns the hash value generated from the given string using the specified hash method.
 - The string “How are you doing?”, for instance, will be hashed into the hash value “60F4A68C292CF0E697C9E65057DCB5B4” (in hexadecimal).
 - Use the online hash calculator at <https://www.pelock.com/products/hash-calculator> to verify the hash value produced by your hashGenerator provider.

Both providers must request the service broker to add their services to the directory, via the `addService()` request. They may remove their respected services from the directory by sending the `removeService()` request to the broker.

4. The **client** (i.e., **service requester**): The client should provide an appropriate user interface allowing the user to specify or choose what the user would want the client program to do; this user input process should continue until the user chooses to exit from the program.
 - a) If the user chooses to get a hash value for a given string using the chosen hashing method, the service requester will first send a `discoverService()` request to the broker in order to find the IP and port# of the service provider; it then sends a message to the provider (with the source string and the hash method as the input parameters) and, in return, gets the hash value back from the provider. Once the hash value is available, the requester then displays it on the user screen.
 - b) If the user chooses to get a random number, the service requester will first send a `discoverService()` request to the broker in order to find the IP and port# of the service provider; it then sends a message to the provider and, in return, gets a random number back from the provider. Once the random number is available, the requester then displays it on the user screen.
 - c) If the user’s request cannot be handled, the requester should print appropriate error messages on the user screen.

Part B

Each team should revise the programs from part A by adding the following enhancements.

1. The service broker will authenticate the provider first before adding the requested service to the directory.
 - For example, the broker may require a provider to register itself first with the broker, with an id and password. If password-based authentication is used to authenticate the provider, the `addService()` and the `removeService()` requests may need to include the provider's id and password as part of the parameters.
2. **(Bonus, 20% more)** In addition to the password-based authentication, you may implement two-factor authentication. That is, in addition to the id and password, the broker may send a one-time code to the requesting provider and require the provider to include that one-time code when requesting to add a service or remove a service.

To hand in:

1. **Part A specification:** Clearly specify the communication protocols between the entities, including those between the broker and the provider, between the broker and the requester, and between the requester and the provider. (Due date: 04.11.2024)
2. **Part A complete programs:** Include the design documents and the complete source programs in a single zip file. Ensure you demo your running program. The demo should be done within a week after the due date. (Due date: 04.11.2024)
3. **Part B complete programs:** Include the design documents and the complete source programs in a single zip file. The demo should be done within a week after the due date. (Due date: 25.11.2024)